

CS2013 - Programación III
Ejercicios: sobrecarga y polimorfismo
Funciones, operadores, Strategy y repositorios polimórficos en C++
Rubén Rivas Medina
Agosto 2026

Indicaciones generales

Duración estimada: 390 minutos. Use C++17, `assert` para las pruebas y nombres de identificadores en inglés. Implemente las validaciones de índices o precondiciones que considere necesarias. No resuelva los ejercicios con variables globales. Cuando se pida un repositorio polimórfico, este debe ser el propietario de los objetos mediante `std::unique_ptr`; no almacene objetos derivados por valor (evita el **object slicing**).

En los ejercicios de sobrecarga de operadores, las asignaciones, los constructores de copia/movimiento y `operator<<` **no cuentan** para el mínimo solicitado. Cada ejercicio declara al menos cuatro sobrecargas que sí cuentan: dos funciones no miembro y dos operadores miembros de clase.

Transferencia a otros escenarios

No trate cada solución como una pieza aislada. Al terminar un ejercicio, identifique qué parte cambiaría si el dominio fuera un videojuego, una aplicación científica, un servicio web o una herramienta de análisis de datos, y qué parte permanecería igual. Las propuestas de **proyección** al final de cada enunciado sirven precisamente para practicar esa reutilización.

Sobrecarga de funciones

Ejercicio 1 (Básico · 20 min): resumen de calificaciones

Contexto. Un sistema académico resume las notas de una evaluación. Según el dato disponible, el promedio puede calcularse para dos, tres o todos los valores de un arreglo. El cliente debe usar el mismo nombre de función sin recordar un nombre distinto para cada caso.

Implemente las tres sobrecargas de `average`. Las versiones de dos y tres argumentos calculan el promedio aritmético. La versión de arreglo recibe un puntero y su tamaño; debe retornar `0.0` si no hay elementos. No use parámetros por defecto para sustituir una sobrecarga.

Idea clave. La firma incluye el nombre y los tipos de parámetros. El tipo de retorno por sí solo no permite distinguir sobrecargas.

```
#include <cassert>
#include <cstdint>

double average(int first, int second);
double average(double first, double second, double third);
double average(const int values[], std::size_t size);
```

Casos de prueba:

```
int grades[] = {14, 16, 18, 20};
assert(average(10, 20) == 15.0);
assert(average(10.0, 15.0, 20.0) == 15.0);
assert(average(grades, 4) == 17.0);
assert(average(grades, 0) == 0.0);
```

Proyección. Reutilice la misma idea para crear `combine` que acepte dos señales, tres señales o un vector de muestras. La sobrecarga permite ofrecer una API uniforme aunque cambie la forma de adquirir datos.

Ejercicio 2 (Intermedio · 30 min): búsqueda por criterios

Contexto. Una biblioteca mantiene sus libros en un `std::vector<Book>`. Un usuario puede buscar por título, por rango de años o por ambos criterios. Las distintas consultas comparten el nombre `findBooks`, pero la colección fuente no debe modificarse.

Implemente las tres sobrecargas de `findBooks`. Todas devuelven un nuevo vector con los libros que cumplen el criterio y conservan el orden original:

- título que contenga el texto dado;
- año entre `fromYear` y `toYear`, ambos inclusive;
- título y rango de años simultáneamente.

Convierta el título y la consulta a minúsculas antes de comparar. Puede crear una función auxiliar privada/no sobrecargada para esa conversión.

```
#include <cassert>
#include <string>
#include <vector>

struct Book {
    std::string title;
    int year;
};

std::vector<Book> findBooks(const std::vector<Book>& books,
                           const std::string& text);
std::vector<Book> findBooks(const std::vector<Book>& books,
                           int fromYear, int toYear);
std::vector<Book> findBooks(const std::vector<Book>& books,
                           const std::string& text,
                           int fromYear, int toYear);
```

Casos de prueba:

```
std::vector<Book> books = {
    {"Clean Code", 2008}, {"Algorithms", 1990}, {"Code Complete", 2004}
};
assert(findBooks(books, "code").size() == 2);
assert(findBooks(books, 2000, 2010).size() == 2);
auto selected = findBooks(books, "code", 2005, 2010);
assert(selected.size() == 1 && selected[0].title == "Clean Code");
assert(books.size() == 3);
```

Proyección. Cambie `Book` por `LogEntry`, `Product` o `Student` sin cambiar la idea de la API. Discuta cuándo varias sobrecargas expresan consultas frecuentes mejor que una función con muchos parámetros opcionales.

Ejercicio 3 (Avanzado · 40 min): distancias entre puntos y trayectorias

Contexto. Un programa de robótica necesita medir distancias en el plano, en el espacio y a lo largo de una trayectoria de puntos. Los tres cálculos son conceptualmente una distancia, pero reciben representaciones diferentes.

Implemente las tres sobrecargas de `distance` y una función auxiliar `segmentLength`. La primera calcula distancia euclidiana 2D, la segunda 3D y la tercera suma las distancias 2D de segmentos consecutivos de una trayectoria. Una trayectoria con cero o un punto mide `0.0`.

Evite duplicar la fórmula 2D: la sobrecarga para trayectorias debe invocar la sobrecarga de dos `Point2D`. Documente en un comentario cuál sobrecarga se selecciona en cada llamada marcada como `// A` y `// B`.

```
#include <cassert>
#include <cmath>
#include <vector>
```

```

struct Point2D { double x; double y; };
struct Point3D { double x; double y; double z; };

double distance(const Point2D& first, const Point2D& second);
double distance(const Point3D& first, const Point3D& second);
double distance(const std::vector<Point2D>& path);
double segmentLength(const Point2D& first, const Point2D& second);

```

Casos de prueba:

```

Point2D a{0, 0}, b{3, 4};
Point3D c{0, 0, 0}, d{1, 2, 2};
std::vector<Point2D> path{{0, 0}, {3, 4}, {3, 8}};
assert(distance(a, b) == 5.0);           // A
assert(distance(c, d) == 3.0);
assert(distance(path) == 9.0);          // B
assert(distance(std::vector<Point2D>{{1, 1}}) == 0.0);

```

Proyección. Estas firmas aparecen en motores gráficos (2D/3D), análisis de GPS y pruebas de trayectorias de robots. Añada como extensión una sobrecarga para una `std::vector<Point3D>` sin duplicar la lógica de acumulación.

Sobrecarga de operadores

Ejercicio 4 (Básico · 30 min): fracciones y desplazamientos 2D

Contexto. Una simulación combina razones exactas (Fraction) y desplazamientos en el plano (Vector2). Las expresiones deben leerse como la notación matemática habitual, sin modificar los operandos cuando se usa una operación binaria.

Implemente las dos clases y, como mínimo, estas **cuatro sobrecargas que cuentan**:

- Funciones no miembro: `operator+` y `operator==` para Fraction.
- Miembros de clase: `Vector2::operator+` y `Vector2::operator+=`.

Normalice toda fracción para que el denominador sea positivo y el numerador y denominador queden reducidos por su máximo común divisor. `operator+` para Fraction debe devolver un nuevo valor y reutilizar `operator==` en una prueba propia. En Vector2, `+` no modifica a los objetos fuente; `+=` sí modifica al objeto izquierdo y devuelve `Vector2&`.

```
#include <cassert>
#include <numeric>

class Fraction {
    int numerator_;
    int denominator_;
    void normalize();
public:
    Fraction(int numerator = 0, int denominator = 1);
    int numerator() const;
    int denominator() const;
};

Fraction operator+(const Fraction& left, const Fraction& right);
bool operator==(const Fraction& left, const Fraction& right);

class Vector2 {
    double x_;
    double y_;
public:
    Vector2(double x = 0.0, double y = 0.0);
    Vector2 operator+(const Vector2& other) const;
    Vector2& operator+=(const Vector2& other);
    double x() const;
    double y() const;
};
```

Casos de prueba:

```
Fraction half(1, 2), quarter(1, 4);
assert(half + quarter == Fraction(3, 4));
assert(Fraction(-2, -4) == half);
Vector2 first(1, 2), second(3, -1);
Vector2 sum = first + second;
assert(sum.x() == 4 && sum.y() == 1);
assert(first.x() == 1 && first.y() == 2);
(first += second) += Vector2(1, 1);
assert(first.x() == 5 && first.y() == 2);
```

Proyección. Fraction es útil en tasas exactas, recetas de recursos o música digital; Vector2 aparece en juegos, interfaces y simuladores físicos. Proponga una operación adicional cuyo significado sea inequívoco antes de sobrecargarla.

Ejercicio 5 (Intermedio · 40 min): polinomios y matrices cuadradas

Contexto. Un módulo de métodos numéricos modela polinomios de grado fijo y matrices cuadradas pequeñas. Debe permitir construir expresiones algebraicas sin exponer los arreglos internos. Los tamaños incompatibles constituyen una precondition inválida: use `std::invalid_argument` cuando corresponda.

Implemente, como mínimo, estas cuatro sobrecargas que cuentan:

- Funciones no miembro: `operator+` y `operator==` para `Polynomial`.
- Miembros de clase: `SquareMatrix::operator*` y `SquareMatrix::operator()`.

`Polynomial` almacena los coeficientes de menor a mayor exponente. Al sumar, el resultado tiene el grado máximo de ambos operandos. `SquareMatrix::operator*` calcula el producto matricial sin modificar los operandos. Las dos versiones de `operator()` (constante y no constante) se consideran una misma operación para el mínimo, pero debe implementar ambas para poder leer y escribir celdas.

```
#include <cassert>
#include <cstdint>
#include <stdexcept>
#include <vector>

class Polynomial {
    std::vector<double> coefficients_;
public:
    explicit Polynomial(std::size_t degree = 0);
    double& coefficient(std::size_t exponent);
    double coefficient(std::size_t exponent) const;
    std::size_t degree() const;
};

Polynomial operator+(const Polynomial& left, const Polynomial& right);
bool operator==(const Polynomial& left, const Polynomial& right);

class SquareMatrix {
    std::size_t size_;
    std::vector<double> data_;
public:
    explicit SquareMatrix(std::size_t size);
    SquareMatrix operator*(const SquareMatrix& other) const;
    double& operator()(std::size_t row, std::size_t column);
    double operator()(std::size_t row, std::size_t column) const;
    std::size_t size() const;
};
```

Casos de prueba:

```
Polynomial p(1), q(2);
p.coefficient(0) = 1; p.coefficient(1) = 2;
q.coefficient(1) = 3; q.coefficient(2) = 4;
Polynomial total = p + q;
assert(total.coefficient(0) == 1 && total.coefficient(1) == 5);

SquareMatrix identity(2), matrix(2);
identity(0, 0) = identity(1, 1) = 1;
matrix(0, 0) = 2; matrix(0, 1) = 3;
matrix(1, 0) = 5; matrix(1, 1) = 7;
assert((identity * matrix)(1, 0) == 5);
```

Proyección. Los polinomios modelan interpolación y animación; las matrices, transformaciones gráficas y análisis de redes. Explique por qué `*` comunica mejor la multiplicación matricial que un nombre genérico como `combine`.

Ejercicio 6 (Avanzado · 50 min): permisos y mapas de píxeles

Contexto. Una aplicación debe combinar permisos de usuario y aplicar filtros de imagen. Ambos dominios son familiares para informática, pero se resuelven con objetos simples y arreglos: no se requiere conocer estructuras de datos ni algoritmos especializados.

Implemente, como mínimo, estas cuatro sobrecargas que cuentan:

- Funciones no miembro: `operator|` y `operator&` para `PermissionMask`.
- Miembros de clase: `GrayImage::operator()` y `GrayImage::operator!`.

`PermissionMask` almacena los bits `Read`, `Write`, `Execute` y `Admin` en un `unsigned int`. `|` combina permisos y `&` conserva los permisos presentes en ambas máscaras. `GrayImage` usa un arreglo dinámico lineal de píxeles; `operator()(row, column)` ofrece acceso de lectura/escritura y `operator!` devuelve una imagen nueva con cada píxel `255 - value`. Valide índices con `std::out_of_range` y aplique la regla de tres a `GrayImage`.

Además de las cuatro sobrecargas mínimas, escriba y justifique una quinta: `PermissionMask::operator==` o `GrayImage::operator==`.

```
#include <cassert>
#include <cstdint>
#include <stdexcept>

class PermissionMask {
    unsigned int bits_;
public:
    enum Permission { Read = 1, Write = 2, Execute = 4, Admin = 8 };
    explicit PermissionMask(unsigned int bits = 0);
    bool has(Permission permission) const;
    // Declare aquí la quinta sobrecarga elegida, si es miembro.
};

PermissionMask operator|(const PermissionMask& left, const PermissionMask& right);
PermissionMask operator&(const PermissionMask& left, const PermissionMask& right);

class GrayImage {
    std::size_t width_;
    std::size_t height_;
    unsigned char* pixels_;
public:
    GrayImage(std::size_t width, std::size_t height);
    GrayImage(const GrayImage& other);
    GrayImage& operator=(const GrayImage& other);
    ~GrayImage();
    unsigned char& operator()(std::size_t row, std::size_t column);
    unsigned char operator()(std::size_t row, std::size_t column) const;
    GrayImage operator!() const;
};
```

Casos de prueba:

```
PermissionMask reader(PermissionMask::Read);
PermissionMask writer(PermissionMask::Write);
PermissionMask combined = reader | writer;
assert(combined.has(PermissionMask::Read));
assert(combined.has(PermissionMask::Write));
assert(!(reader & writer).has(PermissionMask::Read));

GrayImage source(2, 1);
source(0, 0) = 0; source(0, 1) = 120;
GrayImage inverted = !source;
assert(inverted(0, 0) == 255 && inverted(0, 1) == 135);
assert(source(0, 1) == 120);
```

Proyección. Las máscaras aparecen en sistemas de permisos, protocolos y gráficos; las imágenes son una base para filtros de visión. Compare cuándo una sobrecarga es más clara que métodos como `combineWith` o `invert`.

Polimorfismo dinámico y Strategy

En esta sección, una interfaz abstracta representa una familia de algoritmos. El contexto delega el trabajo al objeto estrategia a través de una referencia o puntero a la clase base. Cada repositorio debe almacenar objetos derivados de tipos distintos y permitir recuperarlos mediante su interfaz base. Para no suponer un curso de estructuras de datos, use un arreglo fijo de `unique_ptr` (capacidad cuatro), destructor virtual y `std::move` al registrar una estrategia.

Ejercicio 7 (Básico · 35 min): filtros de calificaciones

Contexto. Un docente aplica distintos criterios para seleccionar notas: un umbral mínimo o solo calificaciones pares. El registro no debe saber los detalles de cada criterio; solo debe delegar en una estrategia intercambiable.

Implemente `GradeFilter` como interfaz abstracta, las estrategias `MinimumGradeFilter` y `EvenGradeFilter`, y `FilterRepository`. El repositorio posee los filtros registrados e identifica cada uno por su `name()`. La clase `GradeReport` recibe los valores y produce una lista filtrada con la estrategia elegida en tiempo de ejecución.

Requisitos de polimorfismo. `accepts` y `name` son virtuales; el destructor de `GradeFilter` es virtual.

`FilterRepository::find` devuelve `const`

`GradeFilter` y lanza `std::out_of_range` si no existe el nombre. No use `if` ni `dynamic_cast` para decidir qué filtro aplicar.

```
#include <array>
#include <cassert>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>

class GradeFilter {
public:
    virtual ~GradeFilter() = default;
    virtual bool accepts(int grade) const = 0;
    virtual std::string name() const = 0;
};

class MinimumGradeFilter : public GradeFilter { /* ... */ };
class EvenGradeFilter : public GradeFilter { /* ... */ };

class FilterRepository {
    std::array<std::unique_ptr<GradeFilter>, 4> filters_;
    std::size_t count_ = 0;
public:
    void add(std::unique_ptr<GradeFilter> filter);
    const GradeFilter& find(const std::string& name) const;
};

class GradeReport {
    std::vector<int> grades_;
public:
    explicit GradeReport(std::vector<int> grades);
    std::vector<int> select(const GradeFilter& filter) const;
};
```

Casos de prueba:

```
FilterRepository repository;
repository.add(std::make_unique<MinimumGradeFilter>(14));
repository.add(std::make_unique<EvenGradeFilter>());
GradeReport report({10, 13, 14, 15, 16});
```

```
assert(report.select(repository.find("minimum-14")) == std::vector<int>({14, 15, 16}));
assert(report.select(repository.find("even")) == std::vector<int>({10, 14, 16}));
```

Proyección. La misma familia de estrategias puede filtrar eventos de un servidor, transacciones sospechosas o sensores fuera de rango. Añada un filtro por intervalo y compruebe que GradeReport no necesita modificación.

Ejercicio 8 (Intermedio · 45 min): formato de mensajes

Contexto. Una aplicación de consola presenta el mismo mensaje con formatos distintos: texto normal o texto en mayúsculas con un prefijo. La pantalla no debe cambiar cuando aparezca un formato adicional; solo recibe una estrategia.

Implemente MessageFormat, PlainFormat, UppercaseFormat y FormatRepository. MessageView es el contexto Strategy: recibe un mensaje y devuelve el texto producido por format. El repositorio es propietario de hasta cuatro formatos mediante un arreglo fijo de std::unique_ptr.

Use std::toupper de <cctype> y convierta a unsigned char antes de llamar a esa función. Registre las estrategias con los nombres "plain" y "upper". No use un if dentro de MessageView para elegir el formato.

```
#include <array>
#include <cctype>
#include <memory>
#include <stdexcept>
#include <string>

class MessageFormat {
public:
    virtual ~MessageFormat() = default;
    virtual std::string name() const = 0;
    virtual std::string format(const std::string& message) const = 0;
};

class PlainFormat : public MessageFormat { /* ... */ };
class UppercaseFormat : public MessageFormat { /* ... */ };

class FormatRepository {
    std::array<std::unique_ptr<MessageFormat>, 4> formats_;
    std::size_t count_ = 0;
public:
    void add(std::unique_ptr<MessageFormat> format);
    const MessageFormat& find(const std::string& name) const;
};

class MessageView {
public:
    std::string show(const std::string& message,
                    const MessageFormat& format) const;
};
```

Casos de prueba:

```
FormatRepository repository;
repository.add(std::make_unique<PlainFormat>());
repository.add(std::make_unique<UppercaseFormat>("ALERTA: "));
MessageView view;
assert(view.show("Servidor listo", repository.find("plain")) == "Servidor listo");
assert(view.show("Servidor listo", repository.find("upper"))
       == "ALERTA: SERVIDOR LISTO");
```

Proyección. El mismo diseño se usa para formatos de fecha, idiomas, temas de interfaz o serialización. Añada TrimFormat o QuotedFormat y compruebe que MessageView no se modifica.

Ejercicio 9 (Intermedio · 45 min): reglas de precio

Contexto. Una tienda aplica diferentes políticas de precio: precio regular, descuento para estudiantes o descuento por cantidad. La orden conoce la cantidad y el precio unitario, pero no debe conocer la fórmula concreta elegida.

Implemente `PriceRule`, `RegularPrice`, `StudentDiscount` y `BulkDiscount`. `Order` es el contexto y delega total a la regla recibida. El repositorio polimórfico `PriceRuleRepository` usa un arreglo fijo de cuatro `unique_ptr`. `BulkDiscount` recibe un mínimo de unidades y un porcentaje; el descuento solo se aplica cuando se alcanza el mínimo. Valide que la cantidad sea positiva y que el porcentaje esté entre 0 y 100.

Agregue una tercera regla creada por usted, por ejemplo una tarifa fija de envío o una promoción de dos artículos por el precio de uno. Demuestre que se registra sin modificar `Order` y escriba un comentario que relacione esto con el principio abierto/cerrado.

```
#include <array>
#include <memory>
#include <stdexcept>
#include <string>

class PriceRule {
public:
    virtual ~PriceRule() = default;
    virtual std::string name() const = 0;
    virtual double calculate(double unitPrice, int quantity) const = 0;
};

class RegularPrice : public PriceRule { /* ... */ };
class StudentDiscount : public PriceRule { /* ... */ };
class BulkDiscount : public PriceRule { /* ... */ };

class PriceRuleRepository {
    std::array<std::unique_ptr<PriceRule>, 4> rules_;
    std::size_t count_ = 0;
public:
    void add(std::unique_ptr<PriceRule> rule);
    const PriceRule& find(const std::string& name) const;
};

class Order {
    double unitPrice_;
    int quantity_;
public:
    Order(double unitPrice, int quantity);
    double total(const PriceRule& rule) const;
};
```

Casos de prueba:

```
PriceRuleRepository repository;
repository.add(std::make_unique<RegularPrice>());
repository.add(std::make_unique<StudentDiscount>(20));
repository.add(std::make_unique<BulkDiscount>(5, 10));
Order small(10.0, 2), large(10.0, 5);
assert(small.total(repository.find("regular")) == 20.0);
assert(small.total(repository.find("student")) == 16.0);
assert(large.total(repository.find("bulk")) == 45.0);
```

Proyección. Las reglas intercambiables modelan promociones, impuestos por región, puntos de fidelidad o cálculo de becas. El patrón evita acumular una cadena difícil de mantener de `if` y `switch` en la clase cliente.

Ejercicio 10 (Avanzado · 55 min): exportación de un reporte de sensores

Contexto. Un programa recoge un pequeño conjunto de mediciones y debe mostrar el mismo reporte como texto legible, CSV o JSON. No se trabaja con archivos ni con estructuras de datos avanzadas: una colección de tamaño fijo basta. El reto está en separar la representación del reporte de las estrategias que lo exportan.

Implemente `ReportExporter`, `TextExporter`, `CsvExporter` y una tercera estrategia propia, `JsonExporter`. `SensorReport` guarda hasta cuatro `Measurement{name, value}` en un arreglo fijo y delega la construcción del texto al exportador. `ExporterRepository` posee exportadores polimórficos y los busca por nombre. Todos deben respetar el orden de registro de las mediciones.

`TextExporter` produce una línea por medición con el formato `name: value`; `CsvExporter` empieza con `name,value` y emplea una línea por medición; `JsonExporter` produce un arreglo JSON sencillo. Restrinja los nombres a letras, números, guion y guion bajo para no requerir escape de comillas. No use condicionales por nombre de formato en `SensorReport`.

```
#include <array>
#include <memory>
#include <stdexcept>
#include <string>

struct Measurement {
    std::string name;
    double value;
};

class ReportExporter {
public:
    virtual ~ReportExporter() = default;
    virtual std::string name() const = 0;
    virtual std::string exportReport(const Measurement* data,
                                    std::size_t count) const = 0;
};

class TextExporter : public ReportExporter { /* ... */ };
class CsvExporter : public ReportExporter { /* ... */ };
class JsonExporter : public ReportExporter { /* ... */ };

class ExporterRepository {
    std::array<std::unique_ptr<ReportExporter>, 4> exporters_;
    std::size_t count_ = 0;
public:
    void add(std::unique_ptr<ReportExporter> exporter);
    const ReportExporter& find(const std::string& name) const;
};

class SensorReport {
    std::array<Measurement, 4> data_;
    std::size_t count_ = 0;
public:
    void add(const std::string& name, double value);
    std::string render(const ReportExporter& exporter) const;
};
```

Casos de prueba:

```
ExporterRepository repository;
repository.add(std::make_unique<TextExporter>());
repository.add(std::make_unique<CsvExporter>());
repository.add(std::make_unique<JsonExporter>());
SensorReport report;
report.add("temperature", 23.5);
report.add("humidity", 60.0);
```

```
assert(report.render(repository.find("text"))
      == "temperature: 23.5\\nhumidity: 60\\n");
assert(report.render(repository.find("csv"))
      == "name,value\\ntemperature,23.5\\nhumidity,60\\n");
```

Proyección. Este patrón permite añadir HTML, XML, una vista de consola o una salida para una API sin tocar la clase que reúne datos. Es la misma separación que aparece entre modelos y vistas en aplicaciones más grandes.

Lista de verificación

- Cada grupo de funciones sobrecargadas se distingue por su lista de parámetros.
- Las operaciones binarias no modifican sus operandos salvo que su nombre y contrato indiquen asignación compuesta, como +=.
- Los ejercicios 4, 5 y 6 contienen al menos dos sobrecargas no miembro y dos sobrecargas miembro de clase, sin contar las operaciones excluidas.
- Todas las interfaces polimórficas tienen destructor virtual y al menos una operación virtual pura.
- Cada repositorio polimórfico posee sus estrategias con `unique_ptr` y expone objetos mediante referencias a la interfaz, sin **slicing**.
- El contexto Strategy depende de la abstracción, no de los algoritmos concretos.